

MATERIAŁ REFERENCYJNY

Databricks Data Engineer Associate

Kompendium do trzydniowego szkolenia

Spis treści

MODUŁ 01	Platforma i workspace	3
MODUŁ 02	Ingestia danych	5
MODUŁ 02A	Lakeflow Connect	7
MODUŁ 03	Delta Lake	9
MODUŁ 05A	Transformacje i modelowanie	11
MODUŁ 04	Optymalizacja Delta i wydajność	14
MODUŁ 05	Ingestia przyrostowa	16
MODUŁ 06	Architektura medalionowa	18
MODUŁ 07	Lakeflow Spark Declarative Pipelines	19
MODUŁ 08	Lakeflow Jobs	22
MODUŁ 09	CI/CD z Git folders i bundle'ami	24
WARSZTAT	Diagnostyka i monitorowanie	26
MODUŁ 10	Nadzór i bezpieczeństwo	28
DODATEK A	Egzamin w pigułce	31

MODUŁ 01

Platforma i workspace

Elementy platformy

Element	Rola
Delta Lake	Otwarty format tabel: pliki Parquet i dziennik transakcji; ACID, time travel, kontrola schematu
Unity Catalog	Nadzór nad danymi i zasobami AI: uprawnienia, lineage, audyt, wyszukiwanie
Lakeflow Connect, Spark Declarative Pipelines, Jobs	Konektory do ingestii; deklaratywne ETL (dawniej Delta Live Tables); orkestracja (dawniej Workflows)
Git folders, Declarative Automation Bundles	Kontrola wersji (dawniej Repos); CI/CD jako kod (dawniej Databricks Asset Bundles)

```

Metastore          jeden na region, podłączony do workspace'ów
├─ Storage credential, external location, connection
├─ Catalog          np. retailhub_jan_kowalski
│   └─ Schema       np. bronze, silver, gold
│       └─ Table, view, materialized view, streaming table, volume, function, model

```

Obiekty mają trzyczłonowe nazwy `catalog.schema.object`. Pliki przechowuje się w wolumenach, `/Volumes/<catalog>/<schema>/<volume>/path`; DBFS root i mounty to rozwiązania legacy.

Zasoby obliczeniowe i modele kosztów

Zasób	Rozliczanie	Najlepszy do	Ograniczenia
Serverless dla notebooków i jobów	DBU za sekundy użycia, infrastruktura w cenie; start w kilka sekund	Domyślny wybór dla notebooków i jobów ETL	Niewiele ustawień Spark, bez init scripts, GPU i Spark UI
Klaster all-purpose (classic)	DBU plus maszyny wirtualne w chmurze przez cały czas działania	Praca interaktywna wymagająca własnej konfiguracji	Płacisz też za bezczynność; ustaw auto-termination
Klaster job (classic)	Niższa stawka DBU dla jobów plus maszyny, tylko w trakcie uruchomienia	Joby harmonogramowane z init scripts, JAR-ami, GPU, instancjami spot	Tworzony dla każdego uruchomienia
SQL warehouse (serverless, pro, classic)	DBU zależne od rozmiaru w czasie działania; auto-stop	Edytor SQL, dashboardy, narzędzia BI przez JDBC/ODBC, wiele równoległych zapytań	Tylko SQL
Serverless dla pipeline'ów	DBU za aktualizacje pipeline'u	Lakeflow Spark Declarative Pipelines	Tylko kod pipeline'u

Sposoby obniżania kosztów klasycznych klastrów: autoskalowanie, auto-termination, instancje spot, cluster policies, Photon. Tryby dostępu: **standard** (dawniej shared, wielu użytkowników) i **dedicated** (dawniej single user).

Co zmienia serverless

Nawyk z klasycznych klastrów	Na serverless
Spark UI	Query profile
Trigger <code>processingTime</code> lub <code>continuous</code> , sink memory lub console	<code>trigger(availableNow=True)</code> do tabeli Delta
Dowolne ustawienia Spark	Tylko kilka, np. <code>spark.sql.shuffle.partitions</code> (domyślnie auto), <code>spark.sql.session.timeZone</code> , <code>spark.sql.ansi.enabled</code>
DBFS root, /tmp	Wolumeny lub pliki workspace
Biblioteki klastra, init scripts	<code>%pip install</code> , środowiska (environments) notebooka lub joba
RDD API, <code>cache()</code> , globalne widoki tymczasowe, notebooki Scala i R	Nieobsługiwane

MODUŁ 02

Ingestia danych

Wzorce ingestii

Wzorzec	Działanie	Narzędzia
Pełny batch	Czyta wszystko i nadpisuje	spark.read, read_files, CTAS
Przyrostowy batch	Przetwarza tylko nowe dane i kończy pracę	COPY INTO, Auto Loader z availableNow, managed connectors
Streaming	Działa w sposób ciągły	Auto Loader, Kafka, streaming tables

Odczyt plików

```
from pyspark.sql.types import StructType, StructField, StringType, DateType

# CSV with a user schema is matched BY POSITION: list every column in file order
customers_schema = StructType([
    StructField("customer_id", StringType(), False), StructField("first_name", StringType()),
    StructField("last_name", StringType()),          StructField("email", StringType()),
    StructField("phone", StringType()),              StructField("city", StringType()),
    StructField("state", StringType()),              StructField("country", StringType()),
    StructField("registration_date", DateType()),    StructField("customer_segment", StringType()),
])
customers = (spark.read.format("csv").option("header", "true")
            .schema(customers_schema)                # one pass, no inference
            .load("/Volumes/<catalog>/default/datasets/customers/customers.csv"))

orders    = spark.read.format("json").load("../orders/orders_batch.json") # JSON: BY NAME
products  = spark.read.parquet("../products/products.parquet")           # schema in the file
```

Opcja	Znaczenie
header, inferSchema	Wiersz nagłówka; wykrywanie typów kosztem dodatkowego przebiegu
sep, nullValue, dateFormat	Parsowanie i konwersja CSV
multiLine	Rekord zajmuje kilka linii (sformatowany JSON, znaki nowej linii w polach CSV)
mode	PERMISSIVE (domyślnie; surowy wiersz w _corrupt_record), DROPMALFORMED, FAILFAST

Przy `header=true` i schemacie podanym przez użytkownika nazwy z nagłówka CSV są ignorowane (`enforceSchema` domyślnie ma wartość `true`). Schemat w złej kolejności łąduje wartości do niewłaściwych kolumn bez żadnego błędu. W JSON pole schematu, którego nie ma w pliku, po cichu przyjmuje wartość `NULL`.

read_files i CTAS

```
CREATE OR REPLACE TABLE bronze.customers AS      -- CTAS: snapshot, a rerun rewrites everything
SELECT *, _metadata.file_path AS source_file, current_timestamp() AS _load_ts
FROM read_files('/Volumes/<catalog>/default/datasets/customers/customers.csv',
               format => 'csv', header => true, schemaHints => 'registration_date DATE');
```

`read_files` wykrywa schemat, wartości niepasujące do schematu umieszcza w `_rescued_data` i czyta formaty csv, json, parquet, avro, orc, text, binaryFile, xml. Metadane pliku są w `_metadata.file_path`, `file_name`, `file_size`, `file_modification_time`; funkcja `input_file_name()` nie działa w Unity Catalog.

```
(df.write.mode("append")                # append | overwrite | error (default) | ignore
 .option("mergeSchema", "true")         # add new columns; overwriteSchema replaces the schema
 .saveAsTable("catalog.bronze.orders"))
```

Pliki lokalne: strona **Add data**, a następnie **Create or modify table**, wczytuje pliki CSV, JSON, Parquet, Avro lub tekstowe do tabeli Delta; można też przesłać je do wolumenu i odczytać przez `read_files`.

Źródła JDBC i REST

```
jdbc_df = (spark.read.format("jdbc")
 .option("url", "jdbc:sqlserver://host:1433;database=sales")
 .option("dbtable", "dbo.orders")                # or .option("query", "SELECT ...")
 .option("user", dbutils.secrets.get("jdbc", "user"))
 .option("password", dbutils.secrets.get("jdbc", "password"))
 .option("partitionColumn", "order_id").option("lowerBound", 1)
 .option("upperBound", 1_000_000).option("numPartitions", 8)
 .load())
jdbc_df.write.mode("append").saveAsTable("catalog.bronze.orders_jdbc")

import requests
resp = requests.get("https://api.example.com/rates", timeout=30); resp.raise_for_status()
dbutils.fs.put(f"/Volumes/{catalog}/bronze/landing/rates/{run_date}.json", resp.text, True)
```

Dane logowania zawsze pobieraj z sekretów. Odczyt przez JDBC jest pełny, chyba że sam filtrujesz po kolumnie znacznika (watermark). Odpowiedzi REST zapisuj jako surowe pliki, ładuj je przez `read_files` lub Auto Loader, a notebook uruchamiaj jako task w Lakeflow Jobs z datami przekazywanymi w parametrach joba.

MODUŁ 02A

Lakeflow Connect

Wybór metody ingestii

Źródło	Pierwszy wybór
Pliki napływające do magazynu w chmurze lub wolumenu	Auto Loader (standard connector) albo <code>read_files</code> w streaming table
Kafka, Kinesis, Pub/Sub, Pulsar	Standard connector w Structured Streaming lub w pipeline
Obsługiwana aplikacja SaaS (Salesforce, Workday, ServiceNow)	Managed connector: bez kodu, przyrostowo, pod nadzorem UC
Obsługiwana baza danych (SQL Server, Azure SQL, MySQL, PostgreSQL, Oracle)	Managed connector z CDC lub change tracking
Źródło bez konektora Databricks	Narzędzie partnerskie (Fivetran, Qlik, Informatica) przez Partner Connect
Jednorazowy eksport, mała tabela słownikowa	JDBC lub REST w notebooku; przyrostowość, ponawianie i zmiany schematu są po Twojej stronie

Kryterium	Wskazuje na
Duży wolumen, miliony plików, częste dostawy	Auto Loader z file events
Zmiany z bazy operacyjnej z małym opóźnieniem	Managed connector do bazy (CDC)
Dane półstrukturalne lub zmienny schemat	Auto Loader albo <code>read_files</code> z <code>_rescued_data</code> , kolumny VARIANT
Centralny nadzór, brak danych logowania w kodzie	Managed connector: connection jest obiektem Unity Catalog
Mały zespół inżynierski	Managed lub partnerski konektor zamiast własnego kodu

Standard connectors (magazyn obiektów, SFTP, kolejki komunikatów) konfiguruje się w kodzie i mogą zapisywać do dowolnej tabeli. **Managed connectors** (aplikacje SaaS, bazy danych) konfiguruje się w UI, przez API lub w bundle'ach; utrzymują streaming tables za pomocą ingestion pipeline.

Managed connector do bazy danych

```
Source database (CDC or change tracking on)
-> 1. Connection          UC object: host, port, credentials
-> 2. Ingestion gateway    classic compute, runs continuously, extracts snapshots and changes
-> Staging volume         UC volume with extracted data
-> 3. Ingestion pipeline   serverless, run on a schedule by a job, applies changes
-> catalog.schema.table    streaming tables, SCD type 1 or 2
```

Konektory SaaS nie potrzebują gatewaya. Gateway musi działać bez przerwy, aby odczytać zmiany, zanim źródło przytnie swój dziennik zmian.

Wymaganie	Szczegóły
Źródło	CDC lub change tracking włączone na tabelach; login o minimalnych uprawnieniach
Uprawnienia	CREATE CONNECTION na metastore; USE CATALOG, USE SCHEMA, CREATE TABLE, CREATE VOLUME na lokalizacji staging i docelowej
Historia	SCD_TYPE_1 (nadpisanie) lub SCD_TYPE_2 (historia; wymaga CDC)

Dane półstrukturalne

Potrzeba	PySpark	SQL
Pole zagnieżdżone, spłaszczenie struktury	<code>col("address.city"), select("item.*")</code>	<code>address.city, item.*</code>
Parsowanie lub odpytanie tekstu JSON	<code>from_json(col("raw"), schema)</code>	<code>from_json(raw, 'city STRING'), raw:a.b</code>
Elastyczne przechowywanie	<code>parse_json(col("raw"))</code> do kolumny VARIANT	<code>parse_json(raw), potem v:customer.city::string</code>
Tablica na wiersze	<code>explode("items"); explode_outer</code> zachowuje puste tablice	<code>explode(items)</code>

MODUŁ 03

Delta Lake

Tabela Delta to pliki danych Parquet i dziennik transakcji w `_delta_log/`; każdy commit dodaje wpis do dziennika, a czytający widzą wyłącznie zatwierdzone wersje. Tabele tworzone w Databricks są domyślnie tabelami Delta.

Modyfikacja danych

```
INSERT INTO silver.customers SELECT * FROM staged_customers;
UPDATE silver.customers SET customer_segment = 'Premium' WHERE customer_id = 'CUST000042';
DELETE FROM silver.customers WHERE customer_id IS NULL;

MERGE INTO silver.customers AS t
USING customer_updates AS s
ON t.customer_id = s.customer_id
WHEN MATCHED AND s.is_deleted THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE;      -- optional: target rows absent from the source
```

MERGE kończy się błędem, gdy jeden wiersz docelowy pasuje do kilku wierszy źródła: najpierw usuń duplikaty klucza w źródle.

SCD typu 2 przez MERGE: staged union

Jeden MERGE nie może jednocześnie zamknąć bieżącego wiersza i wstawić jego nowej wersji dla tego samego klucza, dlatego zmienione wiersze podaje się dwukrotnie: z prawdziwym kluczem (zamyka stary wiersz) i z kluczem NULL (nigdy nie pasuje, więc zostaje wstawiony).

```
MERGE INTO gold.dim_customer AS t
USING (
  SELECT s.customer_id AS merge_key, s.* FROM updates s
  UNION ALL
  SELECT NULL AS merge_key, s.*
  FROM updates s JOIN gold.dim_customer d ON s.customer_id = d.customer_id AND d.is_current
  WHERE s.email <> d.email OR s.city <> d.city
) AS m
ON t.customer_id = m.merge_key AND t.is_current
WHEN MATCHED AND (t.email <> m.email OR t.city <> m.city) THEN
  UPDATE SET is_current = false, end_date = current_date()
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, city, is_current, start_date, end_date)
  VALUES (m.customer_id, m.email, m.city, true, current_date(), NULL);
```

W pipeline lepiej użyć `AUTO CDC ... STORED AS SCD TYPE 2` (Moduł 07).

Schemat, historia i time travel

Mechanizm	Efekt
Kontrola schematu (domyślnie)	Zapis z nieznanymi kolumnami lub niezgodnymi typami kończy się błędem
mergeSchema / overwriteSchema	Dodanie nowych kolumn przy append / zastąpienie schematu przy overwrite
MERGE WITH SCHEMA EVOLUTION INTO	MERGE może dodać kolumny ze źródła
Ograniczenie CHECK, NOT NULL, GENERATED ALWAYS AS IDENTITY	Reguły jakości i klucze zastępcze na poziomie tabeli

```
DESCRIBE HISTORY silver.customers;           -- version, timestamp, operation, user, metrics
DESCRIBE DETAIL silver.customers;           -- location, numFiles, sizeInBytes, clustering
SELECT * FROM silver.customers VERSION AS OF 3;
SELECT * FROM silver.customers TIMESTAMP AS OF '2026-09-01 08:00:00';
RESTORE TABLE silver.customers TO VERSION AS OF 3;           -- itself a new version
VACUUM silver.customers DRY RUN;           -- then VACUUM t [RETAIN 240 HOURS]
```

Właściwość tabeli	Domyślnie	Decyduje o
delta.deletedFileRetentionDuration	7 dni	Jak długo usunięte pliki są przechowywane, zanim VACUUM je skasuje, a więc jak daleko sięga time travel
delta.logRetentionDuration	30 dni	Jak długo przechowywane są wpisy historii

Retencję krótszą niż 7 dni blokuje mechanizm bezpieczeństwa. Po VACUUM wersji wymagających usuniętych plików nie da się już odczytać.

Tabele zarządzane i zewnętrzne, łącznie z konwersją między nimi, omawia Moduł 10.

Typowe błędy Delta

Błąd	Rozwiązanie
Wiele wierszy źródła dopasowanych w MERGE	Usuń duplikaty klucza w źródle
ConcurrentAppendException	Rozdziel operacje równoległe jawnymi warunkami na kolumnach partycjonowania lub klastrowania, wykorzystaj współbieżność na poziomie wierszy (liquid clustering z deletion vectors) albo ponów operację. Ostrzejszy poziom izolacji tego nie naprawi
Brak pliku przy odczycie starej wersji	Odczytaj zachowaną wersję; na przyszłość zwiększ delta.deletedFileRetentionDuration

MODUŁ 05A

Transformacje i modelowanie

Czyszczenie danych z bronze do silver

```
from pyspark.sql import functions as F

silver = (spark.read.table("bronze.orders")
    .filter(F.col("order_id").isNotNull() & F.col("customer_id").isNotNull())
    .withColumn("order_ts", F.to_timestamp("order_datetime"))
    .withColumn("payment_method", F.initcap(F.trim("payment_method")))
    .withColumn("quantity", F.col("quantity").cast("int"))
    .fillna({"discount_percent": 0})
    .dropDuplicates(["order_id"]))
silver.write.mode("overwrite").saveAsTable("silver.orders")
```

Operacja	PySpark	SQL
Usunięcie lub zastąpienie null	dropna(subset=["id"]), fillna({"city": "Unknown"})	WHERE id IS NOT NULL, COALESCE(city, 'Unknown')
Dodanie, zmiana nazwy, usunięcie kolumn	withColumn, withColumnRenamed, drop("c1", "c2")	expr AS new, old AS new, SELECT * EXCEPT (c1, c2)
Rzutowanie, bezpieczne rzutowanie	col("x").cast("int"), F.expr("try_cast(x AS INT)")	CAST(x AS INT), TRY_CAST(x AS INT)
Podział tekstu	F.split("full_name", " ")[0]	split(full_name, ' ')[0]
Filtrowanie	filter(cond) = where(cond)	WHERE
Rozwinięcie tablicy	F.explode("items"), explode_outer, posexplode	explode(items)
Wartość warunkowa	F.when(c, v).otherwise(d)	CASE WHEN c THEN v ELSE d END

Joiny i unie

```
orders.join(customers, "customer_id", "inner")           # one key column in the result
orders.join(customers, ["customer_id", "country"], "left") # multiple keys
orders.join(customers, (orders.cust == customers.customer_id) &
    (orders.region == customers.region), "left")
orders.join(F.broadcast(stores), "store_id")             # force a broadcast join
sizes.crossJoin(colors)                                  # every combination
```

Typ joina	Zwraca
inner / left / right / full	Tylko dopasowania / wszystkie wiersze z lewej / z prawej / z obu stron, NULL tam, gdzie brak dopasowania
left_semi / left_anti	Wiersze z lewej, które mają / nie mają dopasowania; tylko kolumny z lewej
cross	Iloczyn kartezjański

Broadcast join wysyła małą tabelę do każdego executora, dzięki czemu duża tabela nie jest przetasowywana (shuffle). Spark wybiera go automatycznie poniżej `spark.sql.autoBroadcastJoinThreshold` (10 MB); można go wymusić przez `broadcast(df)` lub `/*+ BROADCAST(s) */`.

PySpark	SQL	Dopasowanie kolumn	Duplikaty
<code>a.union(b)</code> , <code>alias unionAll</code>	UNION ALL	Według pozycji	Zachowane
<code>a.union(b).distinct()</code>	UNION	Według pozycji	Usunięte
<code>a.unionByName(b, allowMissingColumns=True)</code>	UNION ALL z kolumnami wymienionymi w tej samej kolejności	Po nazwie; brakujące kolumny dostają NULL	Zachowane

Deduplikacja i agregacje

```
from pyspark.sql.window import Window
w = Window.partitionBy("customer_id").orderBy(F.col("_load_ts").desc())
latest = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")

orders.groupBy("payment_method").agg(
    F.count("*").alias("orders"),
    F.countDistinct("customer_id").alias("customers_exact"),
    F.approx_count_distinct("customer_id").alias("customers_approx"), # HyperLogLog++, rsd 0.05
    F.mean("total_amount").alias("avg_amount"))
orders.select("total_amount").summary().show() # count, mean, stddev, min, quartiles, max
```

Deduplikacja w SQL: `SELECT * FROM t QUALIFY row_number() OVER (PARTITION BY id ORDER BY ts DESC) = 1`. Ranking przy remisach: `row_number` 1, 2, 3; `rank` 1, 2, 2, 4; `dense_rank` 1, 2, 2, 3. `lag` i `lead` zwracają poprzednią i następną wartość w oknie.

Cztery parametry strojenia

Parametr	Domyślnie	Steruje	Gdzie ustawić
<code>spark.sql.shuffle.partitions</code>	200 (classic), auto (serverless)	Liczbą partycji po joinach i agregacjach	<code>spark.conf.set</code> lub <code>SET</code> , także na serverless
<code>spark.default.parallelism</code>	Łączna liczba rdzeni	Liczbą partycji w operacjach RDD	Konfiguracja Spark klastra
<code>spark.executor.memory</code> , <code>spark.driver.memory</code>	Zależnie od typu węzła	Sterują JVM executora lub drivera	Konfiguracja klastra przy tworzeniu; nie w trakcie działania, nie na serverless
<code>spark.sql.autoBroadcastJoinThreshold</code>	10 MB	Maksymalnym rozmiarem tabeli rozgłaszanej automatycznie; -1 wyłącza	<code>spark.conf.set</code> na klasycznym klastrze

Po każdej pojedynczej zmianie mierz ponownie: uruchom tę samą akcję (na przykład `df.write.format("noop").mode("overwrite").save()`), porównaj czas i plan w Spark UI lub query profile. Python UDF są wolne (serializacja wiersz po wierszu); lepiej używać funkcji wbudowanych, SQL UDF lub pandas UDF.

MODUŁ 04

Optymalizacja Delta i wydajność

Układ danych

Technika	Składnia	Zastosowanie
Kompaktowanie	OPTIMIZE t	Wiele małych plików
Liquid clustering	CLUSTER BY (c1, c2), potem OPTIMIZE t	Domyślnie dla nowych tabel; klucze można zmienić bez przepisywania danych
Automatyczne klastrowanie	CLUSTER BY AUTO	Klucze dobierane na podstawie historii zapytań
Partycjonowanie, Z-order (legacy)	PARTITIONED BY (col), OPTIMIZE t ZORDER BY (col)	Starsze tabele; nie łączy się z liquid clustering

```
CREATE TABLE silver.sales (sale_id STRING, customer_id STRING, region STRING, amount DOUBLE)
CLUSTER BY (customer_id, region);
ALTER TABLE silver.sales CLUSTER BY AUTO;      -- or CLUSTER BY (col), CLUSTER BY NONE
OPTIMIZE silver.sales;                          -- incremental; OPTIMIZE ... FULL reclusters all
```

Liquid clustering: maksymalnie 4 klucze; dane są klastrowane przy zapisie powyżej progów rozmiaru, przez OPTIMIZE oraz przez predictive optimization. CLUSTER BY AUTO wymaga tabeli zarządzanej Unity Catalog, predictive optimization i DBR 15.4 LTS lub nowszego. W Pythonie: `df.write.clusterBy("customer_id").saveAsTable("t")`.

Predictive optimization i deletion vectors

Predictive optimization automatycznie uruchamia OPTIMIZE, VACUUM i ANALYZE na **zarządzanych** tabelach Unity Catalog. Jest domyślnie włączona dla kont utworzonych od 11 listopada 2024 r. i stopniowo włączana na starszych kontach.

```
ALTER SCHEMA retailhub.staging DISABLE PREDICTIVE OPTIMIZATION;  -- ENABLE | DISABLE | INHERIT
SELECT * FROM system.storage.predictive_optimization_operations_history;
```

Deletion vectors (domyślnie w nowych tabelach) sprawiają, że DELETE, UPDATE i MERGE oznaczają wiersze jako usunięte zamiast przepisywać pliki Parquet; pliki przepisuje później OPTIMIZE lub REORG TABLE t APPLY (PURGE).

Skew, shuffle i spill

Problem	Objaw w Spark UI lub query profile	Rozwiązanie
Skew (przekos danych)	Maksymalny czas i rozmiar wejścia zadania znacznie powyżej mediany w jednym etapie	AQE skew join (domyślnie włączony); osobna obsługa gorącego klucza; salting kluczy agregacji
Duży shuffle	Duże shuffle read i write; SortMergeJoin z małą stroną	Broadcast małej tabeli; filtrowanie i wybór kolumn przed joinem
Spill	Spill (memory) i spill (disk) powyżej 0	Więcej mniejszych partycji shuffle; mniejszy skew; więcej pamięci na zadanie
Drobne zadania	Tysiące zadań trwających milisekundy w etapie shuffle	Mniej partycji shuffle lub auto
Eksplozja wierszy	Liczba wierszy na wyjściu joina wielokrotnie większa niż na wejściu	Popraw klucz joina

Adaptive query execution (AQE) zmienia plan w trakcie wykonania: łączy małe partycje po shuffle, przełącza się na broadcast join i dzieli partycje z przekosem. Plan sprawdzisz przez `df.explain(mode="formatted")`: `BroadcastHashJoin`, `SortMergeJoin`, `Exchange (shuffle)`, `PushedFilters`. `repartition(n)` wykonuje pełny shuffle; `coalesce(n)` łączy partycje bez pełnego shuffle.

MODUŁ 05

Ingestia przyrostowa

COPY INTO

```
CREATE TABLE IF NOT EXISTS bronze.customers_batch;           -- schemaless target

COPY INTO bronze.customers_batch
FROM (SELECT *, _metadata.file_path AS source_file
      FROM '/Volumes/retailhub/bronze/landing/customers/')
FILEFORMAT = CSV
PATTERN = '*.csv'
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');                        -- needed for a schemaless target
```

Zachowanie	Szczegóły
Idempotentność	Już załadowane pliki są pomijane przy ponownym uruchomieniu, nawet jeśli się zmieniły
Ponowne ładowanie	<code>COPY_OPTIONS ('force' = 'true')</code>
Wybór plików, próba na sucho	<code>FILES = ('a.csv')</code> lub <code>PATTERN; VALIDATE 10 ROWS</code>
Skala	Tysiące plików na uruchomienie; przy milionach lub ciągłym napływie użyj Auto Loadera

Auto Loader

```
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", f"/Volumes/{catalog}/bronze/checkpoints/orders_schema")
  .option("cloudFiles.inferColumnTypes", "true")
  .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
  .load(f"/Volumes/{catalog}/bronze/landing/orders/")
  .select("?", F.col("_metadata.file_path").alias("source_file"))
  .writeStream
  .option("checkpointLocation", f"/Volumes/{catalog}/bronze/checkpoints/orders")
  .trigger(availableNow=True)           # everything new, then stop: incremental batch
  .toTable(f"{catalog}.bronze.orders_autoloader"))
```

Inne opcje: `cloudFiles.schemaHints` (wymuszenie typów kolumn), `cloudFiles.maxFilesPerTrigger`, `cloudFiles.includeExistingFiles`.

Tryb wykrywania plików	Jak znajduje pliki	Zastosowanie
Directory listing (domyślny)	Listuje katalog wejściowy	Prosty start, katalogi średniej wielkości
File notification z file events (zalecany)	<code>cloudFiles.useManagedFileEvents = true</code> ; zdarzenia obsługiwane na external location w UC (włączone file events; DBR 14.3 LTS+)	Duże lub szybko rosnące katalogi
File notification, legacy	<code>cloudFiles.useNotifications = true</code> ; Auto Loader tworzy kolejki w chmurze	Istniejące strumienie

schemaEvolutionMode	Przy nowej kolumnie
addNewColumns (domyślny bez schematu)	Strumień raz kończy się błędem i aktualizuje schemat; po restarcie kolumna jest uwzględniona
rescue	Schemat bez zmian; nieznane kolumny i złe wartości trafiają do <code>_rescued_data</code>
failOnNewColumns	Strumień kończy się błędem aż do naprawy
none (domyślny przy podanym schemacie)	Nowe kolumny są ignorowane

Tryby wyjścia streamingu: **append** (nowe wiersze), **update** (zmienione agregaty), **complete** (cały wynik). Jeden checkpoint na zapytanie; nigdy go nie współdzieli.

MODUŁ 06

Architektura medalionowa

Warstwa	Zawartość	Zapis	Jakość
Bronze	Surowe dane plus <code>_load_ts</code> , <code>source_file</code>	Append	Brak reguł: zachowaj wszystko, z możliwością odtworzenia
Silver	Otypowane, oczyszczone, bez duplikatów, ujednolicone; wymiary SCD1 lub SCD2	MERGE lub overwrite	Reguły egzekwowane, odrzuty w kwarantannie
Gold	Agregaty, fakty i wymiary, KPI	Overwrite lub materialized views	Zaufane dane, nazwy biznesowe

Typ SCD	Działanie	Sygnał w pytaniu
1	Nadpisanie, bez historii	Liczą się tylko bieżące wartości
2	Nowy wiersz przy każdej zmianie, z kolumnami ważności	„Pełna historia”, „wartość z chwili złożenia zamówienia”

Kontrola jakości danych

```
ALTER TABLE silver.orders ADD CONSTRAINT valid_qty CHECK (quantity > 0); -- bad writes fail
```

```
rules = "order_id IS NOT NULL AND customer_id IS NOT NULL AND quantity > 0"
bronze.filter(rules).write.mode("append").saveAsTable("silver.orders")
bronze.filter(f"NOT ({rules}) OR ({rules}) IS NULL") \
    .write.mode("append").saveAsTable("silver.orders_quarantine") # NULL-safe inverse
```

W pipeline te same reguły zapisuje się jako expectations (Moduł 07).

Obiekty warstwy gold w Unity Catalog

Obiekt	Przechowuje dane	Aktualność	Tworzony przez
Tabela	Tak	Gdy zapisze ją job	CTAS, saveAsTable, MERGE
Widok	Nie; zapytanie wykonuje się przy odczycie	Zawsze aktualny	CREATE VIEW
Materialized view	Tak, wynik wyliczony z góry	Przy odświeżeniu, przyrostowo, gdy to możliwe	Pipeline albo CREATE MATERIALIZED VIEW ... SCHEDULE EVERY 1 HOUR na serverless lub pro SQL warehouse
Streaming table	Tak, zasilana dopisywaniem	Każde odświeżenie czyta tylko nowe dane	Pipeline albo CREATE STREAMING TABLE ... AS SELECT * FROM STREAM read_files(...)

MODUŁ 07

Lakeflow Spark Declarative Pipelines

Pipeline to zestaw plików SQL i Python deklarujących zbiory danych. Silnik buduje graf zależności, uruchamia aktualizacje we właściwej kolejności, utrzymuje checkpointy, ponawia operacje, egzekwuje expectations i zapisuje event log. Do zbiorów z tego samego pipeline'u odwołujesz się po nazwie, bez dawnego prefiksu LIVE..

Zbiór danych	SQL	Python (from pyspark import pipelines as dp)	Przetwarzanie
Streaming table	CREATE OR REFRESH STREAMING TABLE	@dp.table zwracający strumieniowy DataFrame	Czyta tylko nowe dane: FROM STREAM(src)
Materialized view	CREATE OR REFRESH MATERIALIZED VIEW	@dp.materialized_view	Utrzymuje poprawny wynik zapytania: FROM src
Widok tymczasowy	CREATE TEMPORARY VIEW	@dp.tmporary_view	Nie jest zapisywany, widoczny tylko w pipeline
Flow	CREATE FLOW ... AS INSERT INTO / AUTO CDC INTO	@dp.append_flow, dp.create_auto_cdc_flow	Zapisuje do streaming table; kilka flow może zasilać jedną tabelę

Deklaracje SQL

```
CREATE OR REFRESH STREAMING TABLE bronze_orders AS
SELECT *, _metadata.file_path AS source_file
FROM STREAM read_files('${order_path}/stream/', format => 'json');

CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_price EXPECT (unit_price >= 0) ON VIOLATION FAIL UPDATE,
  CONSTRAINT has_payment EXPECT (payment_method IS NOT NULL) -- warn only
) AS
SELECT order_id, customer_id, CAST(order_datetime AS TIMESTAMP) AS order_ts,
       quantity * unit_price AS gross_amount
FROM STREAM(bronze_orders);

CREATE OR REFRESH MATERIALIZED VIEW gold_daily_sales AS
SELECT DATE(order_ts) AS order_date, SUM(gross_amount) AS revenue
FROM silver_orders GROUP BY DATE(order_ts);

CREATE OR REFRESH STREAMING TABLE all_orders; -- backfill once, then stream
CREATE FLOW orders_backfill AS INSERT INTO ONCE all_orders BY NAME
  SELECT * FROM read_files('${order_path}/orders_batch.json', format => 'json');
CREATE FLOW orders_stream AS INSERT INTO all_orders BY NAME
  SELECT * FROM STREAM read_files('${order_path}/stream/', format => 'json');
```

Deklaracje w Pythonie

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

@dp.table(name="silver_orders")
@dp.expect_or_drop("valid_order_id", "order_id IS NOT NULL")
@dp.expect_or_fail("valid_price", "unit_price >= 0")
@dp.expect("has_payment", "payment_method IS NOT NULL")
def silver_orders():
    return spark.readStream.table("bronze_orders")          # streaming read -> streaming table

@dp.materialized_view(name="gold_daily_sales")
def gold_daily_sales():
    return (spark.read.table("silver_orders")
            .groupBy(F.to_date("order_datetime").alias("order_date"))
            .agg(F.sum("total_amount").alias("revenue")))
```

Expectation	SQL	Python	Błędne wiersze	Aktualizacja
Ostrzeżenie (domyślnie)	EXPECT (cond)	@dp.expect	Zachowane, zliczone	Trwa dalej
Odrzucenie	ON VIOLATION DROP ROW	@dp.expect_or_drop	Usunięte, zliczone	Trwa dalej
Przerwanie	ON VIOLATION FAIL UPDATE	@dp.expect_or_fail	Nic nie zostaje zapisane	Zatrzymana

@dp.expect_all, expect_all_or_drop i expect_all_or_fail przyjmują słownik reguł. Wzorzec kwarantanny: poprawne wiersze trafiają do silver, a wiersze spełniające odwrotny warunek do tabeli kwarantanny.

AUTO CDC

```
CREATE OR REFRESH STREAMING TABLE silver_customers;

CREATE FLOW silver_customers_cdc AS AUTO CDC INTO silver_customers
FROM STREAM(bronze_customers)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY ingestion_ts
COLUMNS * EXCEPT (operation, _rescued_data)
STORED AS SCD TYPE 2;
```

```
dp.create_streaming_table("silver_customers")
dp.create_auto_cdc_flow(target="silver_customers", source="bronze_customers",
                        keys=["customer_id"], sequence_by=F.col("ingestion_ts"),
                        stored_as_scd_type=2)          # 1 = overwrite in place
```

SCD typu 1 aktualizuje wiersze w miejscu. SCD typu 2 zamyka stary wiersz i dodaje kolumny __START_AT oraz __END_AT (bieżący wiersz: __END_AT IS NULL); TRACK HISTORY ON ogranicza kolumny, których zmiana tworzy nową wersję. SEQUENCE BY porządkuje zdarzenia przychodzące w złej kolejności. Tabela docelowa musi być streaming table.

Uruchamianie i monitorowanie

Ustawienie	Opcje
Tryb	Triggered (przetwarza dostępne dane i kończy) lub continuous
Development / production	Development ponownie używa zasobów i nie ponawia operacji; production ponawia
Odświeżenie	Update (przyrostowo) lub full refresh (reset checkpointów, przebudowa), dla wszystkich lub wybranych tabel
Parametry	Klucze konfiguracji: <code>\${key}</code> w SQL, <code>spark.conf.get("key")</code> w Pythonie

```
SELECT timestamp, details:flow_progress.data_quality.expectations
FROM event_log(TABLE(retailhub.lakeflow.silver_orders)) -- or event_log('<pipeline-id>')
WHERE event_type = 'flow_progress'
AND details:flow_progress.data_quality.expectations IS NOT NULL;
```

Legacy (Delta Live Tables)	Obecnie
<code>import dlt, @dlt.table</code>	<code>from pyspark import pipelines as dp, @dp.table</code>
<code>CREATE LIVE TABLE, FROM LIVE.x, STREAM(LIVE.x)</code>	<code>CREATE OR REFRESH MATERIALIZED VIEW, FROM x, STREAM(x)</code>
<code>APPLY CHANGES INTO, dlt.apply_changes()</code>	<code>AUTO CDC INTO, dp.create_auto_cdc_flow()</code>

MODUŁ 08

Lakeflow Jobs

Taski i sterowanie przepływem

Typ taska	Uruchamia
Notebook, skrypt Python, wheel, JAR, dbt	Kod z parametrami
SQL	Zapytanie, plik SQL lub alert na SQL warehouse
Dashboard	Odświeżenie dashboardu AI/BI, opcjonalnie wysyłkę migawki e-mailem
Pipeline	Aktualizację pipeline'u Lakeflow Spark Declarative Pipelines
Run Job	Inny job
If/else condition	Porównuje dwie wartości; kolejne taski idą gałęzią true lub false
For each	Zagnieżdżony task raz dla każdego elementu listy, z określoną współbieżnością

Ustawienie	Szczegóły
Zależności	depends_on buduje DAG; niezależne taski działają równolegle
Run if	All succeeded (domyślnie), At least one succeeded, None failed, All done, At least one failed, All failed
Statusy	Succeeded, Failed, Timed out, Canceled, Skipped, Upstream failed (nie uruchomiony), Excluded
Ponowienia	Dla taska: max_retries, min_retry_interval_millis, retry_on_timeout; tylko przy błędach przejściowych
Inne	Limity czasu taska i joba, powiadomienia (e-mail, webhook, przekroczenie czasu), max_concurrent_runs

```
tasks:
- task_key: validate
  notebook_task: {notebook_path: ../src/validate}
  max_retries: 2
  min_retry_interval_millis: 60000
- task_key: enough_rows
  depends_on: [{task_key: validate}]
  condition_task: {op: GREATER_THAN, left: "{{tasks.validate.values.row_count}}", right: "100"}
- task_key: check_each_table
  depends_on: [{task_key: enough_rows, outcome: "true"}]
  for_each_task:
    inputs: "{{job.parameters.tables}}"          # JSON array
    concurrency: 2
    task:
      task_key: check_one_table
      notebook_task: {notebook_path: ../src/check, base_parameters: {table: "{{input}}"} }
```

Task values i parametry

```
dbutils.jobs.taskValues.set(key="row_count", value=row_count)           # upstream task
rows = dbutils.jobs.taskValues.get(taskKey="validate", key="row_count",
                                   default=0, debugValue=0)             # downstream task
```

Wartości muszą dać się zserializować do JSON i mieć najwyżej 48 KiB. W ustawieniach tasków używaj `{{tasks.validate.values.row_count}}`, `{{job.parameters.catalog}}`, `{{job.run_id}}`, `{{job.start_time.iso_date}}`, a w `For each {{input}}` `dbutils.notebook.exit()` zwraca wartość do `dbutils.notebook.run()`; nie przekazuje wartości między taskami joba.

Wyzwalacze

Wyzwalacz	Uruchamia job, gdy	Ustawienia
Scheduled	Nadejdzie czas z wyrażenia Quartz cron	Cron, strefa czasowa, wstrzymanie
File arrival	W wolumenie UC lub external location pojawią się nowe pliki	Minimalny odstęp między wyzwoleniami, czas oczekiwania po ostatniej zmianie; sprawdzanie mniej więcej co minutę
Table update	Zmienia się tabele UC (Delta, materialized views, streaming tables, widoki)	Dowolna lub wszystkie tabele, minimalny odstęp, czas oczekiwania po ostatniej zmianie
Continuous	Zakończy się poprzednie uruchomienie	Zawsze aktywne jedno uruchomienie
Manual	Run now, API, CLI	Brak

Harmonogram czasowy pasuje do stałych terminów i źródeł bez zdarzeń. Wyzwalacze sterowane danymi (file arrival, table update) pasują do nieregularnych dostaw: job startuje, gdy dane są dostępne, bez pustych przebiegów.

```
sek min godz dzień-mies. mies. dzień-tyg.
0 0 6 * * ? codziennie o 06:00
0 30 2 ? * MON-FRI w dni robocze o 02:30
0 */15 * * * ? co 15 minut
```

Repair run ponownie wykonuje tylko wybrane nieudane taski i taski od nich zależne, wykorzystując wyniki tych, które się powiodły.

MODUŁ 09

CI/CD z Git folders i bundle'ami

Git folders

Krok	Gdzie
Sklonowanie repozytorium	Workspace, Create, Git folder
Utworzenie lub przełączenie gałęzi, np. feature/jan	Okno Git (przycisk z nazwą gałęzi obok folderu)
Przegląd zmian, commit i push	Okno Git: opis commita, Commit & Push
Otwarcie, przegląd i scalenie pull requesta	Dostawca Git (GitHub, Azure DevOps, GitLab)
Pobranie scalonych zmian do workspace	Okno Git: przełącz na main, Pull

Okno Git obsługuje też merge, rebase, rozwiązywanie konfliktów i reset. Po scaleniu CI waliduje i wdraża bundle.

Budowa bundle'a

```
# databricks.yml
bundle: {name: retailhub}
include: [resources/*.yaml]           # one file per job, pipeline, ...
variables:
  catalog: {description: Target catalog, default: retailhub_trainer}
targets:
  dev:
    mode: development
    default: true
  prod:
    mode: production
  workspace:
    host: https://adb-1234567890123456.7.azuredatabricks.net
    root_path: /Workspace/Shared/.bundle/${bundle.name}/${bundle.target}
    variables: {catalog: retailhub_prod}
    run_as: {service_principal_name: "000000000-0000-0000-0000-000000000000"}
```

```
# resources/retailhub_job.yaml
resources:
  jobs:
    retailhub_job:
      name: retailhub_job
      tasks:
        - task_key: run_pipeline
          pipeline_task: {pipeline_id: "${resources.pipelines.retailhub_pipeline.id"}}
```

Ścieżki względne w pliku zasobu liczone są od tego pliku. Dostępne podstawienia to m.in. `${var.catalog}`, `${bundle.target}` i `${workspace.current_user.userName}`.

Wartość zmiennej pochodzi, w kolejności priorytetu, z: `--var="catalog=retailhub_jan"`, zmiennej środowiskowej `BUNDLE_VAR_catalog`, pliku `.databricks/bundle/<target>/variable-overrides.json`, sekcji `variables:` targetu, a na końcu z default.

	mode: development	mode: production
Nazwy	Z prefiksem [dev <user>]	Takie jak w deklaracji
Harmonogramy i wyzwalacze	Wstrzymane	Aktywne
Tożsamość, ścieżki	Wdrażający użytkownik, jego folder domowy	Service principal przez <code>run_as</code> , wspólny <code>root_path</code>
Kontrole	Łagodne: równoległe uruchomienia, bez blokady	Pipeline'y nie w trybie development; opcjonalna kontrola gałęzi Git

Databricks CLI

```
brew tap databricks/tap && brew install databricks      # macOS, Linux
winget install Databricks.DatabricksCLI                 # Windows
databricks auth login --host https://adb-...azuredatabricks.net --profile TRAINING
databricks bundle validate -t dev --var="catalog=retailhub_jan"
databricks bundle deploy -t dev --var="catalog=retailhub_jan"
databricks bundle run -t dev retailhub_job
databricks bundle destroy -t dev
```

Stary pakiet `pip install databricks-cli` (0.18) nie ma polecenia `bundle`. W CI (np. GitHub Actions) zainstaluj CLI akcją `databricks/setup-cli`, uwierzytelnij service principal zmiennymi `DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID` i `DATABRICKS_CLIENT_SECRET`, a potem uruchom `bundle validate` i `bundle deploy -t prod`. Promocja między środowiskami zmienia wyłącznie konfigurację targetu, nigdy kod.

WARSZTAT

Diagnostyka i monitorowanie

Gdzie szukać

Pytanie	Pierwsze miejsce
Kiedy zaczęło się psuć lub zwalniać?	Zakładka Runs joba: lista uruchomień i widok macierzy czasów tasków
Gdzie wystąpił błąd?	DAG uruchomienia: pierwszy czerwony task; taski Upstream failed nie były uruchamiane
Dlaczego wystąpił błąd?	Wynik taska: pierwsza linia błędu (klasa, ścieżka, tabela, uprawnienie)
Dlaczego działa wolno?	Spark UI (classic) lub query profile (serverless): metryki etapów i operatorów
Dlaczego w ogóle nie wystartował?	Event log zasobu obliczeniowego i logi init scripts
Dlaczego pipeline odrzucił wiersze lub się zatrzymał?	Event log pipeline'u, metryki jakości danych

Porównuj uruchomienia z punktem odniesienia: stopniowo rosnący czas sugeruje przyrost danych bez utrzymania tabel (małe pliki, brak klastrowania); skokowa zmiana tuż po wdrożeniu wskazuje na to wdrożenie; błędy o tej samej godzinie sugerują kolizję z innym obciążeniem. Ustaw na jobie próg ostrzeżenia o czasie trwania, a historię odpytuj w SQL:

```
SELECT run_name, result_state, period_start_time,
       timestampdiff(MINUTE, period_start_time, period_end_time) AS minutes
FROM system.lakeflow.job_run_timeline      -- per task: system.lakeflow.job_task_run_timeline
WHERE period_start_time > current_timestamp() - INTERVAL 30 DAYS
ORDER BY period_start_time DESC;
```

Metryki etapów

Metryka (min, mediana, max w etapie)	Stan zdrowy	Problem
Czas zadania, rozmiar wejścia zadania	Maksimum blisko mediany	Maksimum znacznie powyżej mediany: skew
Shuffle read i write	Proporcjonalne do danych	Bardzo duże przy joinach i agregacjach: shuffle
Spill (memory, disk)	0	Powyżej 0: spill
Liczba zadań	Około rozmiar danych / 128 MB	Tysiące zadań trwających milisekundy: nadmiar partycji
Wiersze na wyjściu joina	Około liczby wierszy wejścia	Wielokrotność wejścia: zły klucz joina

Typowe awarie

Awaria	Objaw	Gdzie patrzeć	Rozwiązanie
OOM executora	Ponawiane zadania, <code>OutOfMemoryError</code> , utracone executory	Spill, rozmiary partycji, broadcast dużej tabeli	Więcej partycji shuffle, usunięcie skew, niższy próg broadcast, większe węzły
OOM drivera	Notebook się odłącza po <code>collect()</code> lub <code>toPandas()</code>	Log drivera, komórka zbierająca dane	Zapis do tabeli, <code>limit()</code> , większy driver
Konflikt bibliotek	<code>ModuleNotFoundError</code> , <code>AttributeError</code> , błędy pip	Event log zasobu, status bibliotek	Przypięte wersje; <code>%pip</code> w notebooku; osobne środowiska jobów
Nieudany start klastra	Błąd bez czasu zadań: init script, brak pojemności, limit	Event log zasobu, logi init scripts	Poprawa init scripta; inny typ węzła lub strefa; pule; serverless
Brak uprawnień	<code>PERMISSION_DENIED ... SELECT on table</code>	Obiekt wskazany w błędzie	Nadanie uprawnień oraz <code>USE CATALOG</code> i <code>USE SCHEMA</code>
Nie znaleziono	<code>TABLE_OR_VIEW_NOT_FOUND</code> , <code>PATH_NOT_FOUND</code>	Pierwsza linia błędu, <code>SHOW TABLES</code>	Poprawa nazwy; dodanie brakującego <code>depends_on</code>
Kolumny po cichu NULL	Ładowanie się udaje, kolumny są puste	Schemat a nazwy w JSON lub kolejność kolumn CSV	Poprawa nazw (JSON) lub kolejności (CSV)

Zasada ogólna: brak czasu zadań oznacza infrastrukturę; błąd przed jakimkolwiek etapem oznacza analizę zapytania lub uprawnień; jeśli etapy się wykonały, szukaj przyczyny w wydajności lub danych.

MODUŁ 10

Nadzór i bezpieczeństwo

Uprawnienia

Uprawnienie	Na	Pozwala na
USE CATALOG, USE SCHEMA	Katalog, schemat	Dostęp do obiektów wewnątrz; potrzebne oprócz uprawnień do obiektu
SELECT, MODIFY	Tabela, widok (lub schemat albo katalog dla wszystkich)	Odczyt; wstawianie, aktualizację, usuwanie
CREATE SCHEMA, CREATE TABLE, CREATE VOLUME, CREATE FUNCTION	Katalog, schemat	Tworzenie obiektów
READ VOLUME, WRITE VOLUME, EXECUTE	Wolumen, funkcja	Pliki; wywołanie funkcji
READ FILES, WRITE FILES, CREATE EXTERNAL TABLE	External location	Bezpośredni dostęp do plików, tabele zewnętrzne
MANAGE	Dowolny obiekt	Nadawanie, odbieranie, usuwanie bez bycia właścicielem
ALL PRIVILEGES	Dowolny obiekt	Wszystkie stosowne uprawnienia oprócz MANAGE

Unity Catalog domyślnie odmawia dostępu. Uprawnienia nadane na katalogu lub schemacie są **dziedziczone** przez obecne i przyszłe obiekty wewnątrz. Nadawać je może właściciel obiektu, właściciel nadrzędnego katalogu lub schematu, posiadacz **MANAGE** albo administrator metastore. Uprawnienia nadawaj **grupom kontowym** (grupy lokalne workspace są odrzucane), a do automatyzacji service principalom (po identyfikatorze aplikacji).

```
GRANT USE CATALOG ON CATALOG retailhub TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA retailhub.gold TO `analysts`;
GRANT SELECT ON TABLE retailhub.bronze.customers TO `analysts`;
REVOKE SELECT ON TABLE retailhub.bronze.customers FROM `analysts`;
SHOW GRANTS ON SCHEMA retailhub.gold;
ALTER TABLE retailhub.gold.daily_sales OWNER TO `data_engineers`;
```

W UI: **Catalog Explorer**, obiekt, **Permissions**, **Grant** lub **Revoke**.

DENY: polecenie SQL DENY działa tylko w starym katalogu **hive_metastore**, gdzie ma pierwszeństwo przed nadaniami. Unity Catalog nie ma DENY dla uprawnień do danych: uprawnienia się sumują, więc dostęp odbiera się przez REVOKE na poziomie, na którym go nadano, przez nadawanie uprawnień na niższym poziomie zamiast na całym schemacie albo przez ograniczenie danych filtrami wierszy, maskami kolumn i ABAC. Polityki ABAC DENY (Beta) mogą odebrać wyłącznie **MANAGE ACCESS CONTROL**.

Filtry wierszy i maski kolumn

```
CREATE OR REPLACE FUNCTION retailhub.gold.mask_email(email STRING)
RETURNS STRING
RETURN CASE WHEN is_account_group_member('admins') THEN email
             ELSE concat('***@', substring_index(email, '@', -1)) END;
ALTER TABLE retailhub.bronze.customers ALTER COLUMN email SET MASK retailhub.gold.mask_email;

CREATE OR REPLACE FUNCTION retailhub.bronze.segment_filter(customer_segment STRING)
RETURNS BOOLEAN
RETURN is_account_group_member('admins') OR customer_segment = 'Premium';
ALTER TABLE retailhub.bronze.customers
  SET ROW FILTER retailhub.bronze.segment_filter ON (customer_segment);

ALTER TABLE retailhub.bronze.customers ALTER COLUMN email DROP MASK;
ALTER TABLE retailhub.bronze.customers DROP ROW FILTER;
```

Funkcja filtra wierszy zwraca **BOOLEAN** (wiersz jest widoczny, gdy wynik to true); maska zwraca typ kolumny. Oba mechanizmy działają przy każdym zapytaniu do tabeli.

Kontrola dostępu oparta na atrybutach (ABAC)

Opiekunowie danych oznaczają tabele i kolumny **governed tags**; **polityka** na katalogu, schemacie lub tabeli stosuje funkcję filtra lub maski do każdego obiektu z pasującymi tagami.

```
ALTER TABLE retailhub.bronze.customers ALTER COLUMN email SET TAGS ('pii' = 'email');

CREATE OR REPLACE POLICY mask_pii_email
ON SCHEMA retailhub.bronze
COLUMN MASK retailhub.gold.mask_email
TO `account users` EXCEPT `admins`
FOR TABLES
MATCH COLUMNS has_tag_value('pii', 'email') AS email_col
ON COLUMN email_col;

CREATE OR REPLACE POLICY premium_only
ON SCHEMA retailhub.bronze
ROW FILTER retailhub.bronze.segment_filter
TO `analysts`
FOR TABLES
WHEN has_tag_value('sensitivity', 'high')
MATCH COLUMNS has_tag('segment') AS seg
USING COLUMNS (seg);

SHOW POLICIES ON SCHEMA retailhub.bronze;
DROP POLICY mask_pii_email ON SCHEMA retailhub.bronze;
```

Nadal działają też formy camelCase `hasTagValue` i `hasTag`. Utworzenie polityki wymaga **MANAGE** (lub własności) na obiekcie i **EXECUTE** na funkcji.

Tabele zarządzane i zewnętrzne

```
CREATE TABLE retailhub.silver.orders (order_id STRING, amount DOUBLE);           -- managed
CREATE TABLE retailhub.bronze.orders_ext (order_id STRING, amount DOUBLE)       -- external
LOCATION 'abfss://landing@retailhubadls.dfs.core.windows.net/orders/';

ALTER TABLE retailhub.bronze.orders_ext SET MANAGED;                          -- external to managed
ALTER TABLE retailhub.bronze.orders_ext UNSET MANAGED;                        -- roll back within 14 days
UNDROP TABLE retailhub.silver.orders;                                         -- within the recovery period (7 days)
```

Czynność	Zarządzana	Zewnętrzna
Utworzenie	Bez LOCATION	LOCATION w obrębie external location; wymaga CREATE EXTERNAL TABLE
Modyfikacja	DML i ALTER TABLE; utrzymanie przez predictive optimization	Ten sam SQL; utrzymanie po Twojej stronie
Usunięcie	Usuwa metadane i dane	Usuwa metadane; pliki zostają
Konwersja	Na zewnętrzną: brak polecenia w miejscu; odtworzenie przez DEEP CLONE ... LOCATION lub CTAS z LOCATION	Na zarządzaną: SET MANAGED (Delta, właściciel tabeli, DBR 17.3 LTS+ lub serverless)

Dostęp do zewnętrznego magazynu buduje się ze **storage credential** (tożsamość w chmurze, np. Azure access connector) i **external location** (ścieżka wraz z tym credentialem).

DODATEK A

Egzamin w pigułce

Pozycja	Wartość
Exam guide	Databricks Certified Data Engineer Associate, 4 maja 2026 r.
Format	45 punktowanych pytań jednokrotnego lub wielokrotnego wyboru (mogą dojść pytania niepunktowane), 90 minut
Próg zaliczenia	Nie jest publikowany; celuj w co najmniej 80% w testach próbnych
Opłata, ważność	200 USD; 2 lata
Kod	Głównie PySpark; SQL przy ingestii, DDL i nadzorze
Doświadczenie	Okolo sześciu miesięcy praktyki inżynierii danych w Databricks

#	Domena	Waga	Moduły
1	Databricks Intelligence Platform	6%	01
2	Data Ingestion and Loading	21%	02, 02a, 05
3	Data Transformation and Modeling	22%	03, 05a, 06, 07
4	Working with Lakeflow Jobs	16%	08
5	Implementing CI/CD	10%	09
6	Troubleshooting, Monitoring, and Optimization	10%	04, warsztat diagnostyczny
7	Governance and Security	15%	10

Częste pułapki

Temat	Pamiętaj
Expectation bez ON VIOLATION	Tylko ostrzeżenie: wiersze zostają, metryki są zapisywane
union w PySpark	Jak SQL UNION ALL: według pozycji, z duplikatami
CSV ze schematem użytkownika	Dopasowanie według pozycji; nazwy z nagłówka ignorowane
Ponowne COPY INTO	Załadowane pliki pomijane, chyba że COPY_OPTIONS ('force' = 'true')
OPTIMIZE a VACUUM	Kompaktuje pliki a usuwa pliki bez odwołań (domyślna retencja 7 dni)
Liquid clustering	Do 4 kluczy; nie łączy się z partycjonowaniem ani Z-order; CLUSTER BY AUTO na tabelach zarządzanych
Predictive optimization	Tylko tabele zarządzane Unity Catalog
Repair run	Ponawia tylko nieudane taski i taski od nich zależne
Wartości między taskami	dbutils.jobs.taskValues, nie notebook.exit
Harmonogram joba	Quartz cron z polem sekund: 0 0 6 * * ?
mode: development	Prefiks [dev user], wstrzymane wyzwalacze
DENY	Brak w Unity Catalog; używaj REVOKE i węższych nadań
Serverless	availableNow, query profile, wolumeny; bez processingTime i Spark UI
Skew	Maksymalny czas zadania znacznie powyżej mediany; AQE skew join